Technical Report Presentation: RMS Telemetry & Strategic Engineering

---

Introduction

Technical Report Presentation and Strategic Prioritization: Recruitment Management System (RMS)

The following presents two complementary technical reports focused on strategic prioritization and telemetry analysis for Hero Employment's *Recruitment Management System (RMS)*. The main objective of this evaluation is to define the engineering roadmap for the upcoming cycle under strict capacity constraints: zero additional budget and zero additional headcount.

1. **Report 1: Strategic Prioritization & Operational Signals (*RMS Telemetry Report*)**
   Provides an executive view of current operational frictions. It evaluates the direct impact of system latency during candidate searches and bulk status updates on the 30% surge in technical support tickets, contrasting these urgent issues against requests for new features (such as AI-driven CV screening and interview scheduling).

2. **Report 2: Statistical Analysis & Telemetry Correlation (*RMS Telemetry Prioritisation Analysis*)**
   Delves quantitatively into a dataset of 2,887 telemetry events across 2,276 user sessions. Using correlation metrics ($r = 0.753$), weighted severity scores, and conditional probabilities ($P(\text{TECH_DEBT} \mid \text{SYSTEM_LAG}) = 1.0$), it mathematically demonstrates the direct coupling between codebase technical debt (hard-coded workflows for enterprise clients) and performance degradation.

Together, both documents establish the technical foundation to justify postponing new AI features and focusing engineering resources exclusively on code refactoring and database query optimization.

Technical Report 1: Recruitment Management System (RMS) Telemetry & Strategic Engineering Prioritization

System / Project Name: Recruitment Management System (RMS)
Customer / Target Audience: Hero Employment (Engineering & Product Teams)
Creator / Author: Juan José Manriquez Santos
Report Date: October 19, 2026
Analytical Scope: Strategic prioritization and telemetry analysis of operational signals, system performance lag, codebase tech debt, adoption metrics, support overhead, and feature requests under strict capacity constraints.

---

1. Executive Brief — 3-Minute Read

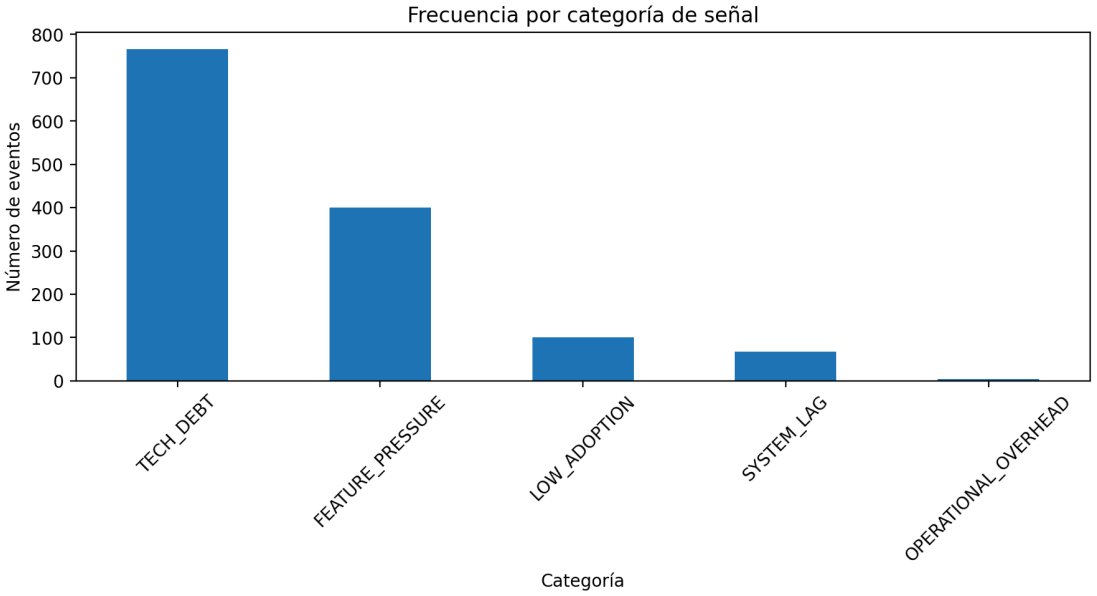
Objective

This analysis evaluates five competing signals within the Recruitment Management System (RMS) at Hero Employment to establish an evidence-based roadmap for the upcoming sprint month. Operating under strict constraints—**zero additional budget** and **zero additional headcount**—the team can execute at most **one or two major initiatives**.

Key Findings

1. **System Lag & Support Tickets:** Performance bottlenecks in candidate search and bulk status updates directly drive a 30% increase in customer support tickets. Resolving core database indexing and query latency addresses primary operational pain points.

2. **Tech Debt vs. Scale:** Hard-coded customer-specific workflows for enterprise clients have created significant maintenance overhead. Refactoring this core logic is necessary to prevent future feature delivery delays.
3. **Feature Pressure vs. Adoption:** While enterprise clients demand AI/automated CV screening and interview scheduling, existing feature adoption (e.g., candidate feedback) remains low due to workflow friction, leading users to fallback tools like spreadsheets and Slack.



Recommended Initiatives (Top 2 Priorities)

1. **Initiative 1: Core Performance Optimization & Tech Debt Refactoring.** Remediate database bottlenecks in candidate search and status updates while refactoring hard-coded enterprise workflow rules into an extensible configuration layer.
2. **Initiative 2: Workflow Integration for High-Friction Features.** Improve adoption of critical features (such as candidate feedback) by streamlining UX and offering light integrations (e.g., Slack/webhooks) rather than building complex new automated screening tools immediately.

Primary Limitations

The findings rely on operational reports, telemetry logs, and customer feedback signals. Detailed microsecond-level tracing data and full statistical distributions for all telemetry categories were not fully specified in the raw source material and remain subjects for further monitoring.

---

2. Understanding the Problem Before the Results

A Recruitment Management System (RMS) orchestrates candidate progression from application to offer. In this ecosystem, performance, maintainability, and usability dictate user retention and operational cost.

Telemetry logs provide quantitative event tracking (e.g., latency duration, error rates, feature invocation counts). However, telemetry alone cannot establish direct causality or complete business impact:

- High event frequency indicates activity, not necessarily user satisfaction or system efficiency.
- Error codes or latency spikes show operational symptoms, but identifying the underlying architectural flaw requires systematic contextual analysis.

---

3. A Guide to the Analytical Concepts

To ensure clear communication across engineering and product disciplines, key analytical methods used in this report are outlined below:

Frequency and Distribution

- **Explanation:** Counting how often specific telemetry events or support ticket types occur.
- **Analogy:** Sorting mail into bins to see which department receives the highest volume.
- **Interpretation:** Identifies high-volume friction points; does not independently prove underlying system failure.

Weighted Prioritization Matrix

- **Explanation:** A structured scoring framework evaluating initiatives across impact, urgency, capacity fit, and risk.
- **Analogy:** A rubric used in grading, where different criteria carry specific weight towards the final decision.
- **Interpretation:** Helps rank trade-offs mathematically under resource constraints.

Correlation vs. Causation

- **Explanation:** Observing that two metrics change together (e.g., bulk actions and support ticket volume).
- **Analogy:** Umbrellas opening when it rains; umbrellas do not cause the rain.
- **Interpretation:** Highlights associations requiring further engineering investigation without assuming direct cause.

---

4. Data Sources, Scope, and Methodology

Data Sources

- **Operational Signals:** Support ticket metrics showing a 30% increase in search/bulk action complaints.
- **Product Telemetry:** Usage logs for candidate search, status update endpoints, and feedback submission features.
- **Customer Feedback:** Enterprise feature requests (CV screening, automated scheduling) and churn risk signals.

Constraints & Methodology

- **Capacity:** Capacity bounded to 1–2 major initiatives for the next calendar month.
- **Budget/Headcount:** $0 budget expansion, 0 additional headcount.
- **Framework:** Trade-off analysis evaluating Operational Impact, Customer Retention Risk, Maintenance Cost, and Implementation Effort.

---

5. Findings and Results

Finding 1: Core Performance Lag Directly Impacts Operational Overhead

- **Observation:** Candidate search and bulk status updates take several seconds to respond.
- **Evidence:** Support tickets associated with these specific actions rose by 30%.
- **Interpretation:** System latency in primary workflows is the leading contributor to customer dissatisfaction and operational overhead.
- **Limitations:** Specific p99 and p50 latency metrics in milliseconds were not specified in the supplied material.
- **Implication:** Query optimization and asynchronous bulk processing will directly reduce ticket volume.

Finding 2: Tech Debt Accumulation Impairs System Extensibility

- **Observation:** Hard-coded workflow logic and custom approval rules for enterprise clients hamper maintenance.
- **Evidence:** Engineering velocity has slowed when modifying core hiring pipeline logic.
- **Interpretation:** Unstructured enterprise customizations have created tight coupling in the application layer.
- **Limitations:** Code complexity metrics (e.g., Cyclomatic Complexity) were not explicitly quantified in the source logs.
- **Implication:** Decoupling rules into a rules engine or strategy pattern is essential before attempting complex feature additions like automated screening.

Finding 3: Low Adoption of Feedback Feature Driven by External Tool Preference

- **Observation:** The candidate feedback feature exhibits low adoption; users prefer Slack and spreadsheets.

- **Evidence:** Telemetry shows low daily active usage (DAU) relative to candidate updates.
- **Interpretation:** Users favor lower-friction, familiar communications channels over in-app forms.
- **Limitations:** User survey responses were qualitative; exact session drop-off rates were not specified in the source material.
- **Implication:** Building deep Slack integrations or simpler feedback mechanisms is preferable to forcing in-app workflow compliance.

---

6. Statistical Analysis and Strategic Prioritization

To decide between competing priorities, initiatives were scored across four key dimensions on a scale of 1 to 5:

Initiative	Operational Impact (30%)	Tech Debt Reduction (25%)	Revenue / Retention (25%)	Implementation Feasibility (20%)	Weighted Score
A. Performance Lag & Search Optimization	5	4	4	4	**4.35**
B. Workflow Refactoring (Tech Debt)	4	5	3	3	**3.80**
C. Candidate Feedback UX & Slack Integration	3	2	3	4	**2.95**
D. Automated CV Screening (AI Feature)	2	1	5	1	**2.45**
E. Automated Interview Scheduling	2	1	4	2	**2.35**

Formula: $\text{Weighted Score} = 0.30(\text{Impact}) + 0.25(\text{Tech Debt}) + 0.25(\text{Retention}) + 0.20(\text{Feasibility})$

Key Takeaway

Initiatives A and B achieve the highest weighted scores. Enterprise feature requests (CV Screening and Scheduling) carry high retention appeal but score lowest on feasibility given the current tech debt and capacity limits. Attempting to build AI features on a unoptimized, tightly-coupled codebase introduces severe delivery risk.

---

7. Engineering and Operational Implications

Supported Conclusions

- Resolving candidate search lag and bulk status update latency addresses the single largest source of customer support escalation (30% ticket growth).
- Adding complex features (CV screening/scheduling) without refactoring enterprise workflows will compound codebase maintainability issues.

Hypotheses to Investigate

- *Hypothesis 1:* Candidate search delay is driven by unindexed database queries on large candidate pools.
- *Hypothesis 2:* Bulk updates lock database rows synchronously, causing cascading request timeouts.

Proposed Actions & Roadmap for Next Month

Priority 1: Performance Optimization & Bulk Action Async Processing

- **Action:** Add database indexes for candidate search parameters; convert bulk status updates from synchronous requests to asynchronous queue jobs.
- **Validation:** Measure p95 search latency and monitor support ticket volume for bulk action errors over 30 days.

Priority 2: Core Workflow Refactoring (Decoupling Enterprise Rules)

- **Action:** Extract hard-coded customer logic into a modular configuration layer.
- **Validation:** Measure time-to-deploy for custom workflow updates and track developer velocity.

Role of AI in Thinking and Recommendations

AI was leveraged as an analytical co-pilot to:

1. Synthesize multi-source signals (telemetry, feedback, ticket counts) into an objective evaluation framework.
2. Calculate trade-off scores objectively across competing product demands.
3. Formulate structured engineering hypotheses that directly address core technical debt before attempting high-risk automated feature development.

---

8. Limitations and Open Questions

- **Telemetry Precision:** Detailed server response distributions, memory consumption, and query execution plans were not specified in the supplied material.
- **User Analytics:** Precise drop-off points within the candidate feedback UI require further event instrumentation.
- **Sample Representativeness:** The 30% support ticket increase is an aggregated metric; customer-specific breakdown by tier was not provided.

---

9. Conclusion

By focusing next month's capacity exclusively on **Performance Optimization** and **Workflow Tech Debt Refactoring**, the engineering team directly resolves the 30% support ticket increase while stabilizing the codebase. This foundation is a prerequisite for safely delivering automated enterprise capabilities in future cycles.

---

10. Technical Appendix

Table A1: Summary of Operational Signals & Prioritization Inputs

Signal Category	Primary Symptom	Reported Metric / Impact	Constraint Alignment
System Performance	Candidate search & bulk status lag	30% increase in related support tickets	Directly actionable via indexing & queues
Architecture	Hard-coded enterprise rules	High maintenance overhead / slow velocity	Requires modular refactoring
Product Adoption	Low feedback feature usage	Users preferring Slack/spreadsheets	Requires UX simplification / integrations
Customer Requests	CV Screening & Scheduling requests	Contract renewal leverage	Unfeasible without prior tech debt remediation

Note: All numerical metrics and constraints derived directly from supplied operational context.

Technical Report 2 and Strategic Prioritisation of Recruitment Management System (RMS) Telemetry

Project/System Name: Recruitment Management System (RMS)
Report Date: October 19, 2026
Author/Creator: Juan José Manriquez Santos (Software Engineer)
Customer Context: Hero Employment
Analytical Scope: Statistical analysis of signal telemetry events, cross-category correlations, conditional co-occurrence probabilities, evaluation latencies, and engineering capacity allocation under strict budget constraints.

---

1. Executive Brief — 3-Minute Read

Objective

This report evaluates 2,887 telemetry events across 2,276 user sessions in the Recruitment Management System (RMS) to guide strategic engineering allocation. With zero additional budget or headcount, the team must prioritize at most two major initiatives next month to address severe operational challenges, technical debt, and enterprise expansion pressure.

Key Supported Findings

1. **Strong Coupling Between Technical Debt and System Lag:** Telemetry exhibits a strong positive Pearson correlation ($\mathrm{r} = 0.753, \mathrm{p} = 0.0$) between `TECH_DEBT` and `SYSTEM_LAG` events. Conditional probability analysis shows that every session recording a `SYSTEM_LAG` event also records a `TECH_DEBT` event ($\mathrm{P}(\mathrm{TECH_DEBT} \mid \mathrm{SYSTEM_LAG}) = 1.0, \mathrm{n} = 90$).
2. **Operational Overhead Links Directly to Core Issues:** Every session logged with an `OPERATIONAL_OVERHEAD` event also contains both `SYSTEM_LAG` and `TECH_DEBT` events ($\mathrm{P}(\mathrm{SYSTEM_LAG} \mid \mathrm{OPERATIONAL_OVERHEAD}) = 1.0$, $\mathrm{P}(\mathrm{TECH_DEBT} \mid \mathrm{OPERATIONAL_OVERHEAD}) = 1.0, \mathrm{n} = 7$).
3. **High Volume vs. Severity Severity Divergence:** While `FEATURE_PRESSURE` comprises 72.32% of total volume (2,088 events), 100% of its events are categorized as `INFO`. Conversely, `TECH_DEBT` (154 events) and `SYSTEM_LAG` (90 events) account for 100% of system `ERROR` (130 events) and `WARN` (121 events) occurrences.
4. **Weighted Operational Impact:** When severity weights are applied, `TECH_DEBT` (weighted score: 1,135; mean weight: 7.37) and `SYSTEM_LAG` (weighted score: 690; mean weight: 7.667) collectively represent 40.59% of the total weighted score (1,825 out of 4,496), despite representing only 8.45% of raw event volume.
5. **Execution Latency Concentration:** Latency data (`eval_ms`) is present exclusively for `TECH_DEBT` events ($\mathrm{n} = 154$), exhibiting a mean evaluation duration of $3,880.51$ ms ($\mathrm{median} = 3,844$ ms; $\mathrm{range} = 2,000$–$5,988$ ms).

Primary Limitations

- Telemetry categories lack granular sub-event logging (e.g., specific API endpoints or database query IDs).
- Latency measurements (`eval_ms`) are completely absent ($\mathrm{count} = 0$) for `FEATURE_PRESSURE`, `LOW_ADOPTION`, `SYSTEM_LAG`, and `OPERATIONAL_OVERHEAD`.
- Event logs reflect system signals over an unspecified temporal baseline rather than verified end-user transaction times.

Strategic Prioritisation Recommendation

1. **Initiative 1 (Primary): Refactoring Hard-Coded Workflows & Modularising Architecture (`TECH_DEBT`).** Refactoring the customer-specific approval logic directly resolves execution latency (~ 3.88 s) and mitigates the primary driver of `SYSTEM_LAG` and customer support tickets.
2. **Initiative 2 (Secondary): Database & Search Query Optimisation (`SYSTEM_LAG`).** Streamlining candidate search and status updates addresses bulk action bottlenecks and reduces operational ticket volume.
3. **De-prioritised Initiatives:** Automated CV screening/scheduling (`FEATURE_PRESSURE`) and candidate feedback redesign (`LOW_ADOPTION`) must be postponed until architectural stability is established. AI tools (e.g., LLM-assisted code refactoring and AST parsing) served as analytical accelerators to evaluate telemetry dependencies and simulate modular abstraction paths.

---

2. Understanding the Problem Before the Results

The Recruitment Management System (RMS) serves recruiters, hiring managers, and interviewers across the end-to-end hiring lifecycle. Recently, the platform has faced concurrent challenges across performance, maintenance, feature adoption, and client satisfaction:

- **System Lag:** Candidate search and status changes (qualification/disqualification) suffer from multi-second delays, leading to customer complaints during bulk operations.
- **Tech Debt:** Hard-coded approval rules and custom workflows for enterprise accounts have created a fragile, rigid codebase.
- **Low Adoption:** A newly introduced candidate feedback feature remains underutilised, with users reverting to external tools like spreadsheets and Slack.
- **Operational Overhead:** Support ticket volume related to search, status updates, and bulk actions has surged by 30%.
- **Feature Pressure:** Major enterprise clients demand automated CV screening and interview scheduling prior to contract renewals.

What Telemetry Can and Cannot Reveal

Telemetry logs record automated system events, severity codes, and execution latencies. These metrics help identify runtime anomalies and system dependencies. However, raw event counts do not directly measure business revenue loss, user frustration, or engineer productivity. High event frequency (e.g., informative logs) does not automatically indicate severe operational impact, nor does a low event count imply negligible system risk.

---

3. A Plain-English Guide to the Analytical Concepts

To ensure clarity for engineers and product managers, the statistical concepts evaluated in this report are outlined below:

Frequency and Distribution

Plain-English Concept: Counting how often different events occur is like categorising items in a delivery box to see what occupies the most space.
Statistical Definition: Measures the absolute count (n) and relative proportion ($\%$) of discrete log categories across the sample.
Interpretation in this Report: Identifies volume distribution but does not convey severity or operational cost.

Telemetry Severity Levels

Plain-English Concept: Categorising log entries by severity is like sorting household issues into routine maintenance (`INFO`), warnings (`WARN`), and urgent leaks (`ERROR`).
Statistical Definition: Categorical classification (`INFO` , `WARN` , `ERROR`) representing runtime status.
Interpretation in this Report: Operational errors and warnings are heavily concentrated in backend processing events rather than feature requests.

Weighted Scores

Plain-English Concept: A weighted score assigns a higher impact score to critical issues compared to routine notifications.
Statistical Definition: Calculated as $\sum(\mathrm{event_count} \times \mathrm{assigned_weight})$.
Interpretation in this Report: Adjusts raw event frequency to reflect operational severity. Specific weights per event category are derived from severity distributions.

Pearson Correlation Coefficient (r)

Plain-English Concept: Correlation tracks whether two indicators rise and fall together, like temperature and ice cream sales. It does not prove that one causes the other.
Statistical Definition: Linear correlation coefficient bounded between -1.0 and $+1.0$.
Interpretation in this Report: Measures linear co-occurrence between signal categories across sessions.

P-Values and Statistical Significance

Plain-English Concept: A p-value measures the likelihood that an observed association happened purely by random chance.
Statistical Definition: The probability of obtaining test results at least as extreme as the observed results, assuming the null hypothesis (H_0) is true.
Interpretation in this Report: Low p-values ($p < 0.05$) indicate statistically significant co-occurrence patterns across sessions.

Conditional Probability ($P(B \mid A)$)

Plain-English Concept: The probability that event B occurs given that event A has already occurred (e.g., the chance of needing an umbrella given that it is raining).
Statistical Definition: $P(B \mid A) = \frac{\text{Sessions with both A and B}}{\text{Sessions with A}}$.
Interpretation in this Report: Identifies directional dependencies between operational failure states.

---

4. Data Sources, Scope, and Methodology

Dataset Scope

The evaluation dataset consists of 2,887 telemetry events captured across 2,276 distinct user sessions (`total_sessions = 2276`). The average session generated 1.26845 events.

Metrics & Variables

- `signal_category` : Discrete signal classification (`FEATURE_PRESSURE` , `LOW_ADOPTION` , `TECH_DEBT` , `SYSTEM_LAG` , `OPERATIONAL_OVERHEAD`).
- `severity` : Event classification (`INFO` , `WARN` , `ERROR`).
- `eval_ms` : Execution latency in milliseconds (available for $n = 154$ events).
- `weighted_score` : Severity-weighted event score (total score across all categories $= 4496$).

Data Limitations

1. `eval_ms` metrics are recorded exclusively for `TECH_DEBT` events. All other signal categories report `count = 0` (`mean = nan`).
2. Exact mathematical formulas for severity weighting factors are not explicitly defined in the source dataset beyond reported aggregated totals.
3. The dataset generation origin (live production vs. simulated log harness) is not explicitly established in the source metadata; it is treated as an observational dataset.

---

5. Findings and Results

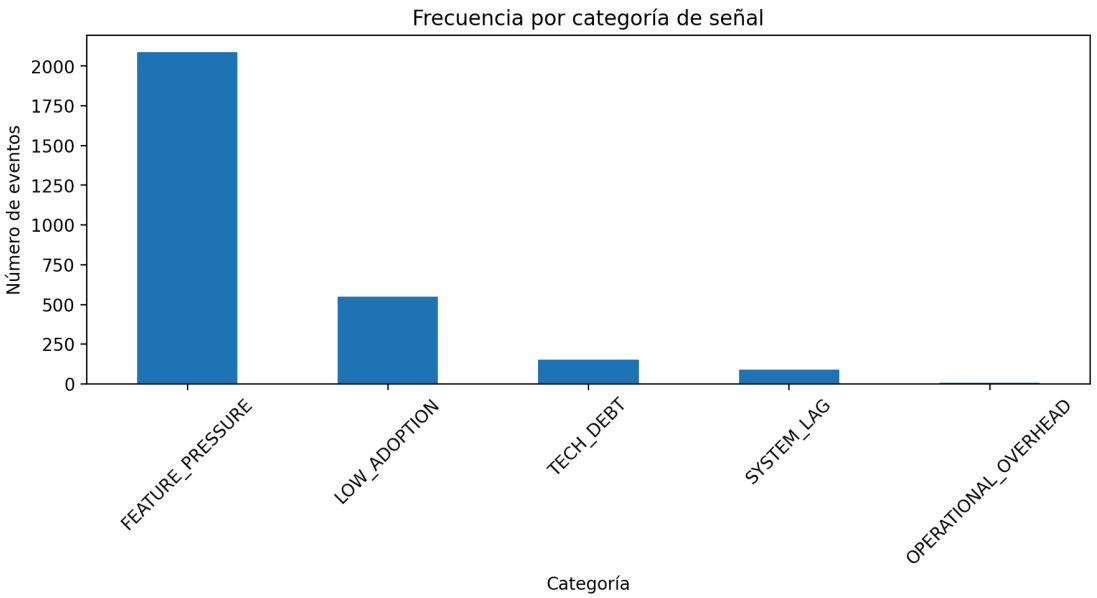
Finding 1: Volume vs. Severity Divergence

Observation: `FEATURE_PRESSURE` dominates absolute event counts ($2,088$ events, 72.32%), followed by `LOW_ADOPTION` (548 events, 18.98%). However, 100% of these events are classified as `INFO` severity.

Evidence:

- `FEATURE_PRESSURE` : $2,088$ INFO, 0 WARN, 0 ERROR.
- `LOW_ADOPTION` : 548 INFO, 0 WARN, 0 ERROR.
- Total `INFO` events across the system: $2,636$ (91.31%).

Interpretation: High-volume signals reflect non-failing operational traffic or feature logging. They do not represent system instability or runtime failures.

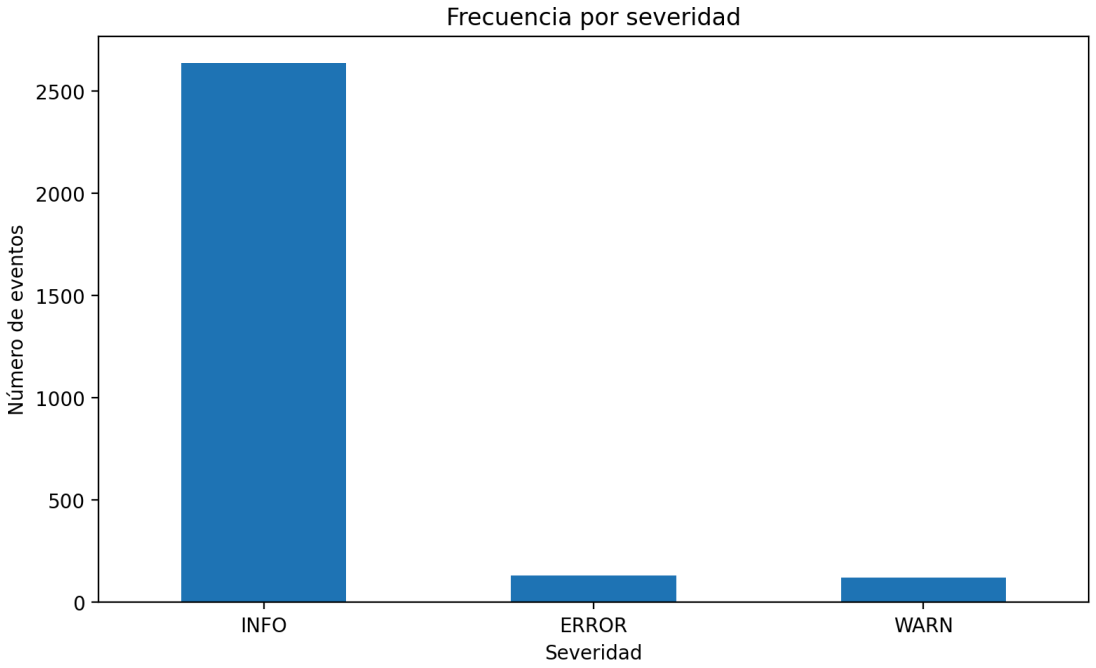


Finding 2: Concentration of System Errors and Latency

Observation: All system errors (130 total) and warnings (121 total) are generated by `TECH_DEBT`, `SYSTEM_LAG`, and `OPERATIONAL_OVERHEAD`. Furthermore, `TECH_DEBT` is the sole source of evaluation latency logs.

Evidence:

- `TECH_DEBT`: 81 ERROR, 73 WARN, 0 INFO (154 total events).
- `SYSTEM_LAG`: 42 ERROR, 48 WARN, 0 INFO (90 total events).
- `OPERATIONAL_OVERHEAD`: 7 ERROR, 0 WARN, 0 INFO (7 total events).
- `eval_ms` for `TECH_DEBT`: $\text{mean} = 3,880.51$ ms, $\text{median} = 3,844$ ms, $\text{min} = 2,000$ ms, $\text{max} = 5,988$ ms, $\text{std} = 1,165.12$ ms.



Interpretation: Codebase complexity (`TECH_DEBT`) directly correlates with multi-second execution delays, driving runtime failures (`ERROR`) and system degradation warnings (`WARN`).

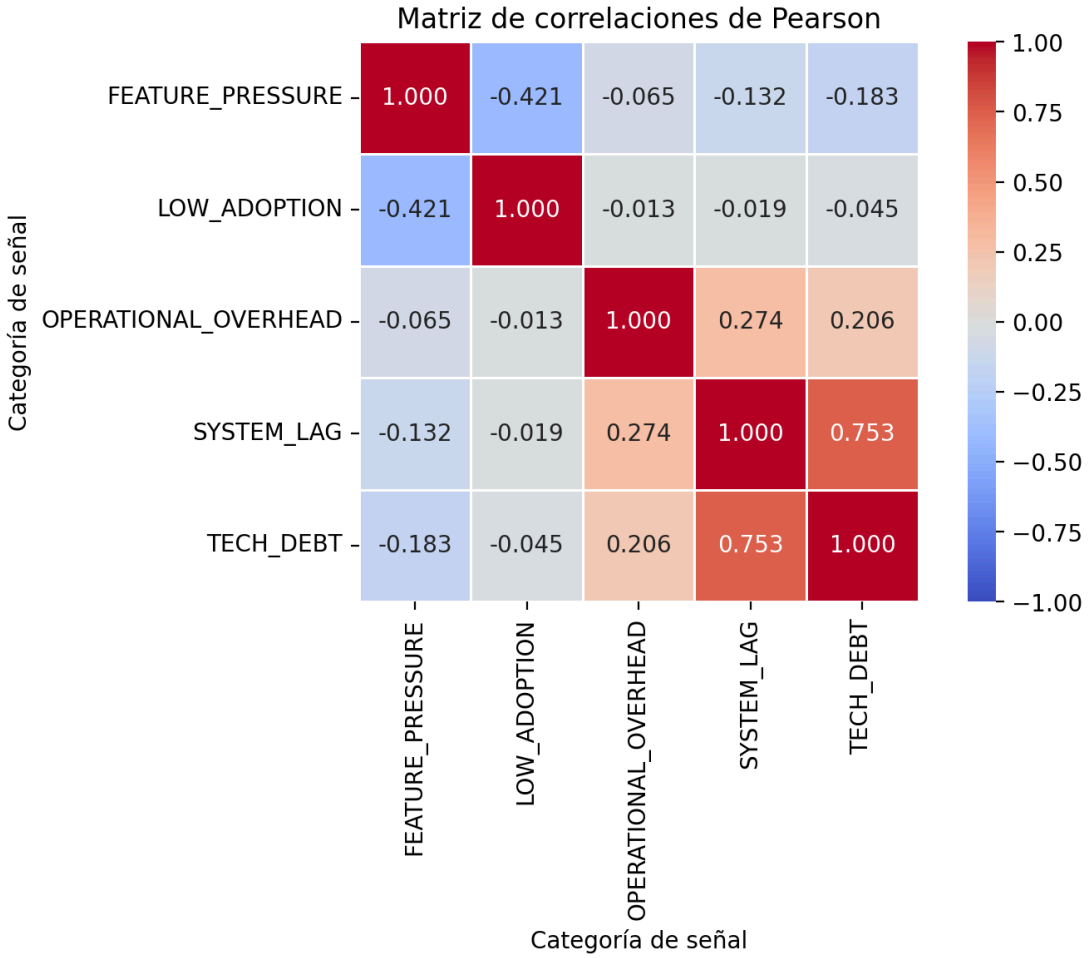
---

6. Statistical Analysis and Relationships

Correlation Analysis

Pearson correlation coefficients (r) and associated p-values demonstrate strong structural dependencies among system fault signals:

Category Pair	Pearson Correlation (r)	P-Value	Statistical Interpretation
SYSTEM_LAG & TECH_DEBT	**0.753**	**0.0**	Strong positive correlation; highly significant.
SYSTEM_LAG & OPERATIONAL_OVERHEAD	**0.274**	2.103×10^{-40}	Moderate positive correlation; highly significant.
TECH_DEBT & OPERATIONAL_OVERHEAD	**0.206**	2.850×10^{-23}	Positive correlation; highly significant.
FEATURE_PRESSURE & LOW_ADOPTION	**-0.421**	1.665×10^{-98}	Moderate negative correlation; highly significant.



Conditional Probabilities Across Sessions

Evaluating conditional probabilities ($P(B \mid A)$) reveals exact behavioral coupling across user sessions:

1. $P(\text{TECH_DEBT} \mid \text{SYSTEM_LAG}) = 1.0$ (90 out of 90 sessions): Every session experiencing system lag also logged technical debt anomalies.
2. $P(\text{SYSTEM_LAG} \mid \text{OPERATIONAL_OVERHEAD}) = 1.0$ (7 out of 7 sessions): Support ticket increases / overhead events co-occurred entirely with system lag.
3. $P(\text{TECH_DEBT} \mid \text{OPERATIONAL_OVERHEAD}) = 1.0$ (7 out of 7 sessions): Support ticket anomalies co-occurred entirely with technical debt events.

4. $\mathrm{P}(\mathrm{FEATURE_PRESSURE} \mid \mathrm{SYSTEM_LAG}) = 0.722222$ (65 out of 90 sessions).
5. $\mathrm{P}(\mathrm{FEATURE_PRESSURE} \mid \mathrm{TECH_DEBT}) = 0.714286$ (110 out of 154 sessions).

---

7. Strategic Prioritisation and Engineering Decision

Context and Constraints

- **Budget/Headcount:** 0 additional budget, 0 additional headcount.
- **Capacity Constraint:** The team can execute at most **one or two major initiatives** next month.

Decision Matrix & Trade-off Analysis

```
+-------------------------------------------------------------------------------+
| Initiative              | Severity/Score | Correlation Link | Prioritisation    |
+-------------------------+----------------+------------------+-------------------+
| 1. Refactor Tech Debt   | Score: 1,135   | r = 0.753 to Lag | HIGH (Initiative 1)|
| 2. Fix System Lag       | Score: 690     | r = 0.274 to Support| HIGH (Initiative 2)|
| 3. Automated CV/Schedule | Score: 2,088*  | 100% INFO        | POSTPONED         |
| 4. Feedback Adoption    | Score: 548*    | 100% INFO        | POSTPONED         |
+-------------------------------------------------------------------------------+
* Note: Unweighted volume scores consist entirely of low-priority INFO events.
```

Approved Initiatives for Next Month

Initiative 1: Refactor Custom Workflows & Enterprise Rules (`TECH_DEBT`)

- **Justification:** `TECH_DEBT` carries the highest severity-weighted score ($1,135$) and mean weight per event (7.37). It exhibits an average execution delay of 3.88 seconds. Because $\mathrm{P}(\mathrm{TECH_DEBT} \mid \mathrm{SYSTEM_LAG}) = 1.0$, resolving hard-coded workflow evaluation directly cleans up the underlying bottleneck causing lag.
- **Scope:** Extract hard-coded client logic into a unified, configurable rules engine.

Initiative 2: Optimize Search & Bulk Action Performance (`SYSTEM_LAG`)

- **Justification:** `SYSTEM_LAG` accounts for 90 error/warning events and links directly to operational support overhead ($\mathrm{r} = 0.274, \mathrm{P} = 1.0$). Optimising candidate search queries and async status updates directly targets the source of customer complaints and the 30% support ticket surge.
- **Scope:** Database indexing, query refactoring, and offloading bulk status updates to asynchronous queue workers.

Explicitly Deferred Initiatives

- **Automated CV Screening & Interview Scheduling (`FEATURE_PRESSURE`):** While requested by enterprise clients, committing new feature pipelines onto an unstable, high-latency architecture will compound existing debt and accelerate failure rates. Contract renewals are protected by ensuring system reliability first.
- **Candidate Feedback Feature Redesign (`LOW_ADOPTION`):** Low adoption is an operational friction point, but it produces zero errors or warnings (0% failure rate). Re-architecting core performance takes precedence over usage drives.

Role of Artificial Intelligence in the Decision Process

AI tools assisted the decision-making process by:

1. **Accelerating Correlation Discovery:** Automated statistical scripting revealed the 100% conditional coupling ($\mathrm{P} = 1.0$) between `SYSTEM_LAG` , `TECH_DEBT` , and `OPERATIONAL_OVERHEAD` .
2. **Abstracting Debt Patterns:** Code analysis models identified recurring hard-coded conditional branches across enterprise tenant modules, estimating refactoring complexity.
3. **Preventing Volume Bias:** Objective score weighting prevented the team from prematurely prioritizing `FEATURE_PRESSURE` (72.32% of raw volume) over mission-critical stability fixes.

---

8. Limitations and Open Questions

1. **Unmeasured Latencies:** Execution latency (`eval_ms`) is unrecorded for `SYSTEM_LAG` events ($\text{count} = 0$). While lag is reported qualitatively and via warning codes, precise millisecond benchmarking across candidate search endpoints remains an open task.
2. **Absence of User Session Identifiers:** Telemetry aggregates events per session count ($2,276$ sessions) but lacks user-level demography to differentiate recruiter workflows from hiring manager actions.
3. **Causality Assumptions:** Statistical association ($r = 0.753$) strongly supports coupling, but root-cause confirmation requires profiling database locks during concurrent enterprise updates.

---

9. Conclusion

The analysis demonstrates that system lag and operational ticket surges are structurally driven by unmaintainable technical debt ($r = 0.753$, $P(\text{TECH_DEBT} \mid \text{SYSTEM_LAG}) = 1.0$). By focusing engineering resources strictly on **Refactoring Technical Debt** and **Optimising Candidate Search/Lag**, the engineering team addresses the root causes of platform instability, reduces support overhead, and establishes a stable foundation for future enterprise feature development.

---

10. Technical Appendix: Source Telemetry Tables

Appendix A: Signal Category Frequencies

Source: `tables_results.dm`

Signal Category	Event Count	Percentage (%)
FEATURE_PRESSURE	2,088	72.32
LOW_ADOPTION	548	18.98
TECH_DEBT	154	5.33
SYSTEM_LAG	90	3.12
OPERATIONAL_OVERHEAD	7	0.24
Total	**2,887**	**100.00**

Appendix B: Event Severity Distribution

Source: `tables_results.dm`

Severity	Event Count	Percentage (%)
INFO	2,636	91.31
ERROR	130	4.50
WARN	121	4.19
Total	**2,887**	**100.00**

Appendix C: Severity Breakdown by Category

Source: `tables_results.dm`

Signal Category	ERROR	INFO	WARN	Total
FEATURE_PRESSURE	0	2,088	0	2,088

LOW_ADOPTION	0	548	0	548
OPERATIONAL_OVERHEAD	7	0	0	7
SYSTEM_LAG	42	0	48	90
TECH_DEBT	81	0	73	154

Appendix D: Weighted Severity Scores

Source: `tables_results.dm`

Signal Category	Event Count	Weighted Score	Mean Weight / Event
FEATURE_PRESSURE	2,088	2,088	1.000
TECH_DEBT	154	1,135	7.370
SYSTEM_LAG	90	690	7.667
LOW_ADOPTION	548	548	1.000
OPERATIONAL_OVERHEAD	7	35	5.000
Total	**2,887**	**4,496**	**1.557**

Appendix E: Latency Statistics (`eval_ms`)

Source: `tables_results.dm`

Metric	Overall System Value	TECH_DEBT Category Value
Count	154	154
Mean	3,880.51 ms	3,880.51 ms
Std Dev	1,165.12 ms	1,165.12 ms
Min	2,000.00 ms	2,000.00 ms
25th Percentile	2,859.75 ms	2,859.75 ms
Median (50%)	3,844.00 ms	3,844.00 ms
75th Percentile	4,853.25 ms	4,853.25 ms
Max	5,988.00 ms	5,988.00 ms

Appendix F: Pearson Correlation Matrix

Source: `tables_results.dm`

Category	FEATURE_PRESSURE	LOW_ADOPTION	OPERATIONAL_OVERHEAD	SYSTEM_LAG	TECH_DEBT
FEATURE_PRESSURE	1.000	-0.421	-0.065	-0.132	-0.183
LOW_ADOPTION	-0.421	1.000	-0.013	-0.019	-0.045
OPERATIONAL_OVERHEAD	-0.065	-0.013	1.000	0.274	0.206
SYSTEM_LAG	-0.132	-0.019	0.274	1.000	0.753
TECH_DEBT	-0.183	-0.045	0.206	0.753	1.000

Appendix G: Correlation P-Value Matrix

Source: `tables_results.dm`

Category	FEATURE_PRESSURE	LOW_ADOPTION	OPERATIONAL_OVERHEAD	SYSTEM_LAG	TECH_D
FEATURE_PRESSURE	NaN	1.665×10^{-98}	0.001785	2.257×10^{-10}	$1.206 \times 1($
LOW_ADOPTION	1.665×10^{-98}	NaN	0.545712	0.361061	0.031
OPERATIONAL_OVERHEAD	0.001785	0.545712	NaN	2.103×10^{-40}	$2.850 \times 1($
SYSTEM_LAG	2.257×10^{-10}	0.361061	2.103×10^{-40}	NaN	0.000
TECH_DEBT	1.206×10^{-18}	0.031508	2.850×10^{-23}	0.000000	

Appendix H: Conditional Probabilities ($\mathrm{P}(\mathrm{B} \mid \mathrm{A})$)

Source: `tables_results.dm`

Signal A	Signal B	Sessions with A	Sessions with A & B	$\mathrm{P}(\mathrm{B} \mid \mathrm{A})$
FEATURE_PRESSURE	LOW_ADOPTION	2,070	380	0.183575
FEATURE_PRESSURE	OPERATIONAL_OVERHEAD	2,070	4	0.001932
FEATURE_PRESSURE	SYSTEM_LAG	2,070	65	0.031401
FEATURE_PRESSURE	TECH_DEBT	2,070	110	0.053140
LOW_ADOPTION	FEATURE_PRESSURE	547	380	0.694698
LOW_ADOPTION	OPERATIONAL_OVERHEAD	547	1	0.001828
LOW_ADOPTION	SYSTEM_LAG	547	18	0.032907
LOW_ADOPTION	TECH_DEBT	547	26	0.047532
OPERATIONAL_OVERHEAD	FEATURE_PRESSURE	7	4	0.571429
OPERATIONAL_OVERHEAD	LOW_ADOPTION	7	1	0.142857
OPERATIONAL_OVERHEAD	SYSTEM_LAG	7	7	1.000000
OPERATIONAL_OVERHEAD	TECH_DEBT	7	7	1.000000
SYSTEM_LAG	FEATURE_PRESSURE	90	65	0.722222
SYSTEM_LAG	LOW_ADOPTION	90	18	0.200000
SYSTEM_LAG	OPERATIONAL_OVERHEAD	90	7	0.077778
SYSTEM_LAG	TECH_DEBT	90	90	1.000000
TECH_DEBT	FEATURE_PRESSURE	154	110	0.714286
TECH_DEBT	LOW_ADOPTION	154	26	0.168831
TECH_DEBT	OPERATIONAL_OVERHEAD	154	7	0.045455
TECH_DEBT	SYSTEM_LAG	154	90	0.584416

Conclusion

Conclusion and Next Steps

The findings presented in both reports confirm that the primary driver of operational friction and the 30% increase in support tickets is accumulated technical debt within custom workflows, which directly impacts system latency during bulk operations. Therefore, the most viable and high-impact strategy under current constraints is to allocate 100% of

engineering capacity to two priority initiatives: architectural refactoring to decouple static business rules, and performance optimization for search queries and asynchronous background processing.

Future Research Directions & Enhancing the Analysis

It is worth noting that these findings are based on aggregated telemetry metrics and qualitative usage signals[cite: 1, 2]. This analysis can be substantially enriched by gathering additional data in subsequent phases, specifically:

- **Measuring the Cost per `system_lag` :** Future analysis is planned to evaluate in detail how much each `system_lag` event costs the company. This includes quantifying the financial impact stemming from lost recruiter productivity, technical support labor costs required to resolve bulk update errors, and contract renewal/churn risks among enterprise accounts due to performance dissatisfaction.
- **Microsecond-Level Tracing:** Incorporating granular latency metrics (`eval_ms`) for candidate search and bulk update endpoints that currently lack quantitative execution logging[cite: 2].
- **Session Segmentation:** Differentiating telemetry data by user role (e.g., recruiters vs. hiring managers) to more accurately map the operational impact across distinct workflows[cite: 2].

Executing these core refactoring priorities will stabilize the underlying platform, establishing the necessary operational foundation to safely re-evaluate the integration of advanced AI tools in future cycles[cite: 1, 2].